Computer Security and privacy
Prof. Thomas Bourgeat

Luigi Ieracitano

September 16, 2026

Contents

1 Introduction to Computer Security **2**
1.1 Why a course on computer security? . . . 2
1.2 What is computer security? . . . 2
1.2.1 Traditional properties: the CIA triad . . . 2
1.2.2 Not-so-traditional (but important) properties . . . 3
1.3 The security policy . . . 3
1.4 The adversary and the threat model . . . 3
1.4.1 Key vocabulary to talk about the adversary . . . 4
1.5 Securing a system: security mechanisms . . . 4
1.5.1 Composing multiple mechanisms . . . 5
1.5.2 Humans are part of the system . . . 5
1.6 How do we show systems are secure? . . . 5
1.7 Security engineering . . . 6
1.8 Summary . . . 6

Chapter 1
Introduction to Computer Security

1.1 Why a course on computer security?

When designing "classical" systems or programs, we usually aim to guarantee three properties:

- **Correctness**: for a given input, the program provides the expected output.
- **Safety**: well-formed programs cannot produce bad (even dangerous) outputs.
- **Robustness**: the system copes with errors, both in the input and during execution.

In all these cases, we try to anticipate what could go wrong and take it into account in the design. Computer security differs from these disciplines by one extra ingredient: the presence of an **adversary**, i.e., an entity that *actively and intelligently* tries to make the system fail, as opposed to a random fault or accidental error.

1.2 What is computer security?

Definition 1 (Computer security)**.** Properties of a computer system must hold **in the presence of a resourced strategic adversary**.

Two elements are therefore central to any security analysis:

1. **Which properties** must be guaranteed?
2. **Against which adversary** (with which capabilities) must they be guaranteed?

1.2.1 Traditional properties: the CIA triad

Definition 2 (Confidentiality)**.** Prevention of unauthorized disclosure of information.
Example: the adversary should not be able to read my bank statement.

Definition 3 (Integrity)**.** Prevention of unauthorized modification of information.
Example: the adversary should not be able to change my bank balance.

Definition 4 (Availability)**.** Prevention of unauthorized denial of service.
Example: the adversary should not be able to prevent me from accessing my bank account.

These three properties form the **CIA triad** (*Confidentiality, Integrity, Availability*), which must be known by heart: it comes up in almost every exercise in this course.

1.2.2 Not-so-traditional (but important) properties

Modern security goes well beyond the CIA triad. Notable examples include:

- **Authenticity**: guaranteeing that a piece of information or an entity is indeed what it claims to be.
- **Anonymity**: preventing the identification of the principal behind an action.
- **Isolation**: preventing one component of the system from affecting another.
- **Non-repudiation**: preventing a principal from denying having performed an action.

Remark 1. These properties will be revisited and formalized in later chapters (authentication, anonymity, etc.). The list above is not exhaustive — there are many more complex properties relevant to modern security.

1.3 The security policy

To precisely define a security property, we must specify *what* it applies to. Two notions are introduced for this:

Definition 5 (Assets / objects)**.** Anything with value (data, files, memory, etc.) that needs to be protected.

Definition 6 (Principals / subjects)**.** People, computer programs, services, etc. that interact with the system. A principal **may not** contain the adversary.

Definition 7 (Security policy)**.** A high-level description of the security properties that must hold in the system, in relation to assets and principals.

Property	**General definition**	**Example (principals / assets)**
Confidentiality	Prevention of unauthorized disclosure of information	authorized users may read a file
Integrity	Prevention of unauthorized modification of information	authorized programs may write a file
Availability	Prevention of unauthorized denial of service	authorized services can access a file

Table 1.1: Examples of security properties expressed in terms of principals and assets.

1.4 The adversary and the threat model

Definition 8 (Threat model)**.** Describes the resources available to the adversary and their capabilities (observe, influence, corrupt, …).

A key point of the course: the adversary is **strategic**. This means they will choose *the best possible way* to use their resources to mount an attack that violates the security properties — one should never assume the adversary will act sub-optimally.

Remark 2. Threat modelling is a **very hard task** in practice: underestimating the adversary's capabilities invalidates the security argument, while overestimating them can make the system

impossible or too costly to build.

1.4.1 Key vocabulary to talk about the adversary

Term	Definition	Examples
Threat	The feared event, the goal of the adversary that we don't want to see materialize	A hacker wants to retrieve money by breaking into the bank's system; a student wants to learn my password by looking over my shoulder
Threat model	Technical term defining the adversary's capabilities	The adversary can observe my connection; the adversary can corrupt my machine; the adversary controls a bank employee
Harm	The bad thing that happens when the threat materializes	The adversary steals money; the adversary blocks access to the bank; the adversary learns my password; the adversary reads the message
Vulnerability	A specific weakness that could be exploited by adversaries interested in a variety of different assets	The banking API is not protected; the password appears in plain text on my screen

Table 1.2: Basic vocabulary for describing an adversary.

Remark 3 (Putting it all together)**.** Computer security means guaranteeing that properties (defined by the **security policy**) hold in the presence of an adversary (described by the **threat model**). If the properties hold within that threat model, it means there is no exploitable **vulnerability** that could materialize a **threat**, and hence no **harm** occurs.

1.5 Securing a system: security mechanisms

Definition 9 (Security mechanism)**.** A technical mechanism used to ensure that the security policy is not violated by an adversary within the threat model.

A security mechanism can combine:

- **Software** (programs),
- **Hardware**,
- **Maths** (cryptography),
- **Distributed systems**, **people**, and **procedures**.

Remark 4. Security mechanisms can be *engineered*, just like any other part of a system.

Example 1 — Policy: ensure the log of transactions is not tampered with by a single employee. Mechanism: keep a copy of the log on multiple computers, such that no single employee has access to all of them.

Example 2 — Policy: ensure messages cannot be read by anyone but the sender and the receiver. Mechanism: encrypt the message before sending it.

1.5.1 Composing multiple mechanisms

Real systems are large and often require **several** security mechanisms combined together. There are two fundamentally different ways of composing them, and distinguishing them clearly is crucial:

Definition 10 (Defence in depth)**.** The security policy is maintained **as long as at least one** mechanism remains effective ("OR" composition).

Definition 11 (Weakest link)**.** The security policy is violated **if any one** mechanism fails ("AND" composition).

Remark 5. Important point for exercises: **security does not necessarily increase linearly with the number of mechanisms**. Depending on how mechanisms are composed, adding one can either strengthen the system (defence in depth) or introduce a new point of failure (weakest link) — e.g., a poorly designed mechanism added "in series" can weaken an otherwise solid system.

1.5.2 Humans are part of the system

Humans are an integral part of the system, and are often directly targeted by the adversary:

- **Phishing** attacks and **social engineering**.
- Bad use of passwords: **weak** passwords, passwords **written down**, or passwords **repeated** across services.

Remark 6. The fact that humans are often the "weakest link" does **not** mean the rest of the system should be neglected: all mechanisms must remain robust.

1.6 How do we show systems are secure?

A very important, asymmetric way of reasoning about security:

- **Attacker**: it is enough to find **one single** way to violate **one single** security property (within the threat model) to "win".
- **Defender**: must show that **no** adversary strategy can violate the security policy.

This asymmetry is what makes security hard: the defender must cover *every* case, while the attacker only needs *one* success.

> "One of the major problems right now is script kiddies. These are people who just download open source tools that are meant for good, and they point them at whatever they want, press 'Go,' and it fires a suite of exploits at a system hoping one of them will work." — Richard Moore, Security Specialist (IBM)

Definition 12 (A "secure" system)**.** A system is said to be **secure** if an adversary constrained by a specific *threat model* **cannot** violate the security policy.

Remark 7. The security of a system is therefore never absolute: it is always **relative to a given threat model**. A system may be secure against a weak adversary and no longer secure against a stronger one.

Definition 13 (Security argument)**.** A rigorous argument (verbal or mathematical) that the security mechanisms in place are indeed effective in maintaining the security policy, subject to the

assumptions of the threat model.

Remark 8 (For a threat model to be useful)**.** The model **must constrain** the adversary — otherwise no security argument is possible (an adversary with unlimited capabilities can always break the system).

Remark 9. Security arguments for the **composition** of several mechanisms are particularly hard to get right (cf. *defence in depth* vs *weakest link* above).

1.7 Security engineering

Designing a secure system typically follows three steps:

1. **High-level specification**
   - Define the architecture of the system (e.g., a high-level block diagram).
   - Define the security policy (principals, assets, security properties).
   - Define the threat model.
2. **Security design**
   - Select / design security mechanisms.
   - State the security argument: which controls maintain which properties?
3. **Secure implementation**
   - Implement the mechanisms.
   - Ensure they conform to the design model.
   - Security testing.

Remark 10. Following this order (specify *before* implementing) is essential: a mechanism added without a clear threat model and security argument risks protecting against nothing precise, or creating a false sense of security.

1.8 Summary

- Security problems always involve an **adversary**.
- The adversary is **strategic**: they will take the most damaging approach possible.
- The adversary's **capabilities** define a **threat model**.
- **Security mechanisms** aim at fulfilling a **security policy** within a given threat model.
- Showing that a system is secure requires providing a **security argument**.
- **Composition** of mechanisms is not trivial (*defence in depth* vs *weakest link*).
- Security is always **relative** to the adversary considered.